

Lecture 7: Reinforcement Learning

Satchit Chatterji

SIAS Group, Ivl, Universiteit van Amsterdam

(based on a slideset by Jelle Zuidema & Iris Proff)

Machine Learning and Language Models, 2026

What is Reinforcement Learning?

Q-learning

RL in the brain

Model-based RL

Extensions of Q-learning

Deep RL

Types of learning algorithms

- **Supervised learning:** learning from labeled examples, such as neural networks
- **Unsupervised learning:** finding patterns in raw data, such as clustering
- **Reinforcement learning:** learning from reward signals

A structural model from behavioral psychology

Pavlov, 1897 Pavlovian learning / classical conditioning
Agent learns a stimulus–outcome contingency

Thorndike, 1905 Law of effect / instrumental conditioning
Agent learns an action–outcome contingency

Tolman, 1930 Latent learning / goal-directed learning
Agent learns a cognitive map

Structural model

- Classical conditioning
- Instrumental conditioning
- Goal-directed learning

Cognitive model

- Rescorla–Wagner model (1972)
- Temporal-difference learning (Samuel, 1959)
- Q-learning (Watkins & Dayan, 1992)
- Model-based learning

$$U(S) = R(S)$$

*The value of a state
depends on the reward it
provides.*

*State-transition
diagram from source
slide 10*

*State-transition diagram from
source slide 11*

Bellman equation

$$U(S) = R(S) + \gamma \max_A [U(S')]$$

U utility

S state

R reward

A action

γ temporal discount
factor

*State-transition diagram from
source slide 11*

Bellman equation

$$U(S) = R(S) + \gamma \max_A [U(S')]$$

U utility

S state

R reward

A action

γ temporal discount
factor

Question 1

Consider states S_2 and S_3 (which can you reach with action A_1 and A_2) and state S_4 (which you can reach from state S_3 by action A_1). How valuable are each of these states relative to eachother? Is $U(S_1) = U(S_3)$?

*State-transition diagram from
source slide 11*

Bellman equation

$$U(S) = R(S) + \gamma \max_A [U(S')]$$

U utility

S state

R reward

A action

γ temporal discount
factor

*State-transition diagram from
source slide 11*

Bellman equation

$$U(S) = R(S) + \gamma \max_A [U(S')]$$

U utility

S state

R reward

A action

γ temporal discount
factor

Question 2

How do you avoid having to first learn everything about the environment/problem before you can start acting in the world?

What is Reinforcement Learning?

Q-learning

RL in the brain

Model-based RL

Extensions of Q-learning

Deep RL

Q-learning and temporal-difference learning

How do we learn the utility of state–action pairs by trial and error?

Learning the Q-value

$$Q(S, A) = \hat{U}(S, A)$$

Q-learning approximates utility through online learning: the policy is applied and learned at the same time.

Q-learning update rule

Prediction error

$$\delta = R + \gamma \max_{A'} Q_t(S', A') - Q_t(S, A)$$

Q-value update

$$Q_{t+1}(S, A) = Q_t(S, A) + \alpha \delta$$

Asymptotic relationship

$$\lim_{t \rightarrow \infty} Q_t(S, A) = U(S, A)$$

Reminder: $U(S, A) = R(S, A) + \gamma \max_{A'} [U(S', A')]$.

State-transition diagram from source slide 13

Q-learning action-selection policy

Suppose we are in state S , with possible actions A_1 and A_2 , and

$$Q(S, A_1) = 0.5, \quad Q(S, A_2) = 0.7.$$

Which action should we choose?

Q-learning action-selection policy

Suppose we are in state S , with possible actions A_1 and A_2 , and

$$Q(S, A_1) = 0.5, \quad Q(S, A_2) = 0.7.$$

Which action should we choose?

Question 3

How do you make sure you maximize reward, both in the short term (taking advantage of your knowledge) and in the long term (extending your knowledge)?

Exploration–exploitation trade-off

**Exploit what you
know**

or

Explore new options

An agent has to exploit what it has experienced in order to obtain reward, but it also has to explore in order to make better action selections in the future.

Sutton & Barto (2018)

Exploration–exploitation trade-off

**Exploit what you
know**

or

Explore new options

An agent has to exploit what it has experienced in order to obtain reward, but it also has to explore in order to make better action selections in the future.

Sutton & Barto (2018) Classic example: [multi-armed bandit](#)

Multi-armed bandit



The name comes from imagining a gambler at a row of slot machines ("one-armed bandits"), who has to decide which machines to play, how many times to play each machine and in which order to play them, and whether to continue with the current machine or try a different machine.

Q-learning action-selection policies

Suppose $Q(S, A_1) = 0.5$ and $Q(S, A_2) = 0.7$. How should we trade off exploration and exploitation?

Epsilon-greedy

$$P(a_i | S) = \begin{cases} 1 - \epsilon & \text{if } q_i = \max_A q_i(A), \\ \frac{\epsilon}{|A| - \max_A q_i(A)} & \text{otherwise.} \end{cases}$$

The best action is chosen with probability $1 - \epsilon$.

Softmax

$$P(a_i | S) = \frac{e^{q_i/\tau}}{\sum_A e^{q_i/\tau}}$$

For $\tau \rightarrow 0$ the policy is greedy; for $\tau \rightarrow \infty$ it is random.

Q-learning pseudo-code

1. Set $\alpha \in [0, 1]$, small $\epsilon > 0$, and $\gamma > 0$.
2. Initialize $Q(s, a)$ for all $s \in S$ and $a \in A$ to 0.
3. Initialize s .
4. For each episode:
 - 4.1 While s is not terminal:
 - 4.1.1 Choose a from s using the action-selection policy.
 - 4.1.2 Observe reward R and next state s' .
 - 4.1.3 Update Q and set $s \leftarrow s'$.

What is Reinforcement Learning?

Q-learning

RL in the brain

Model-based RL

Extensions of Q-learning

Deep RL

Temporal-difference learning in the brain

*Dopaminergic-neuron recording
chart from source slide 18*

- Recording from a single dopaminergic neuron in a monkey brain
- Upper blue: expected reward
- Red: expected reward omitted
- Green: unexpected early or late reward
- Lower blue: expected reward

Hollerman (1998)

What is Reinforcement Learning?

Q-learning

RL in the brain

Model-based RL

Extensions of Q-learning

Deep RL

Model-based versus model-free learning

Model-free (habitual)

Rely on trial and error.

Efficient, but inflexible.

Model-based (goal-directed)

Build a model of the environment. Effortful, but flexible.

Dayan & Niv (2008)

Environment model

$$M : (s, a) \mapsto (s', r)$$

What does my environment look like?

State-transition diagram from source slide 20

Sutton & Barto (2018)

1. Initialize parameters.
2. For each episode, while s is not terminal:
 - 2.1 Choose a from s using the action-selection policy.
 - 2.2 Observe R and s' , update Q , and set $s \leftarrow s'$.
 - 2.3 Update the model: $M(s, a) = (R, s')$.
 - 2.4 Repeat n times:
 - 2.4.1 Choose (s, a) to simulate.
 - 2.4.2 Simulate the action and observe R and s' .
 - 2.4.3 Update Q .

Direct RL update and model learning use real experience; planning uses simulated experience.

Policy during episode 2

Dyna-Q policy diagram from source slide 22

Sutton & Barto (2018)

What is Reinforcement Learning?

Q-learning

RL in the brain

Model-based RL

Extensions of Q-learning

Deep RL

Extensions of RL algorithms

- **Eligibility traces:** update not only the last state–action pair, but all previously visited pairs with a decay factor λ .
- **Decaying learning rate:** α decreases over time.
- **Hybrid models:** combine model-based and model-free components.

What is Reinforcement Learning?

Q-learning

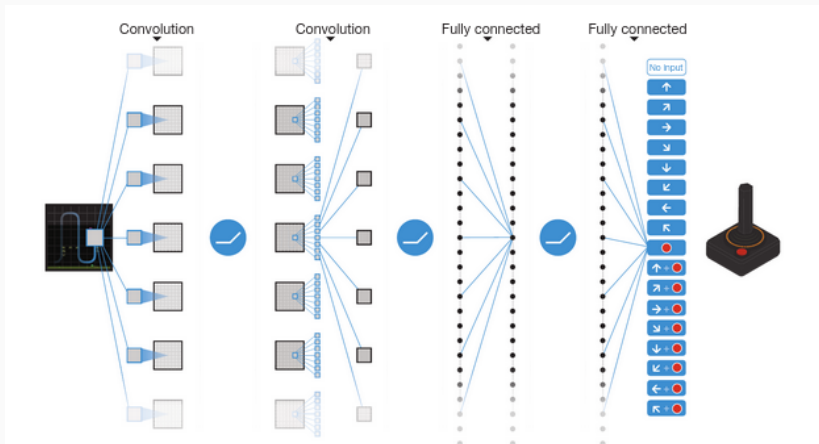
RL in the brain

Model-based RL

Extensions of Q-learning

Deep RL

Deep Reinforcement Learning



Deep Q-Learning (Mnih et al. 2015)

- Replace Q-table with a neural network Q , with weights θ ; use it to compute Q-value of current state
- Make a copy of the Q-network (the *target-network* $\hat{Q}$); use it to compute Q-value of target states
- Learn in batches: act in the world (choose actions using ϵ -greedy) and gather data
- Optimize Q-network (minimize prediction error) via gradient descent on a batch of observations
- Occasionally copy Q into $\hat{Q}$

Deep Q-Learning (Mnih et al. 2015)

Algorithm 1: deep Q-learning with experience replay.

Initialize replay memory D to capacity N

Initialize action-value function Q with random weights θ

Initialize target action-value function $\hat{Q}$ with weights $\theta^- = \theta$

For episode = 1, M **do**

 Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$

For $t = 1, T$ **do**

 With probability ϵ select a random action a_t

 otherwise select $a_t = \operatorname{argmax}_a Q(\phi(s_t), a; \theta)$

 Execute action a_t in emulator and observe reward r_t and image x_{t+1}

 Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

 Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in D

 Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from D

 Set $y_j = \begin{cases} r_j & \text{if episode terminates at step } j+1 \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-) & \text{otherwise} \end{cases}$

 Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ with respect to the network parameters θ

 Every C steps reset $\hat{Q} = Q$

End For

End For

LETTER

Human-level control through deep reinforcement learning

Volodymyr Mnih¹, Koray Kavukcuoglu², David Silver¹, Andrew A. Rusu¹, Joel Veness¹, Marc G. Bellemare¹, Alex Graves¹, Martin Riedmiller¹, Andrew S. Pridmore¹, Georg Ostrovski¹, Sogou Ouyang¹, Charles Beattie¹, Amir Sadik¹, Koray Kavukcuoglu², Ickho Song¹, Tharanitharan Kumar¹, Daan Wierstra¹, Shane Legg¹ & Demis Hasselbraun¹

The theory of reinforcement learning provides a normative account, deeply rooted in psychological and neuroscientific perspectives on animal behaviour, of how agents can optimise their control of an environment. To use reinforcement learning successfully in situations approaching real-world complexity, however, agents are confronted with a difficult task: they must derive efficient representations of the environment from high-dimensional sensory inputs, and use these representations to generalise their experience to novel situations. Remarkably, humans and other animals seem to solve this problem through a harmonious coexistence of reinforcement learning and hierarchical sensory processing systems¹, the former evidenced by a wealth of neural data revealing notable parallels between the phasic signals emitted by dopaminergic neurons and temporal-difference reinforcement learning algorithms². While reinforcement learning agents have achieved success in a variety of domains³, their applicability has previously been limited to domains in which useful features can be hand-engineered, or to domains with fully observed, low-dimensional state spaces. Here we use recent advances in training deep neural networks⁴ to develop a novel artificial agent that learned a deep Q network, that can learn successful policies directly from high-dimensional sensory inputs using end-to-end reinforcement learning. We tested this agent on the challenging domain of classic Atari 2600 games⁵. We demonstrate that the deep Q network agent, receiving only the pixels and the game score as inputs, was able to learn the same policies as previous algorithms and achieve a level comparable to that of a professional human player across sixteen out of 49 games, using the same algorithm, network architecture and hyperparameters. This work bridges the divide between high-dimensional sensory inputs and actions, resulting in the first artificial agent that is capable of learning to excel at a diverse array of challenging tasks.

We set out to create a single algorithm that is able to develop a wide range of competencies in a varied range of challenging tasks – a central goal of general artificial intelligence⁶ that has eluded previous efforts^{7,8}. To achieve this, we developed a novel agent, a deep Q network (DQN), which is able to combine reinforcement learning with a class of artificial neural networks⁹ known as deep neural networks. Notably, recent advances in deep neural networks⁴, in which several layers of nodes are used to build up progressively more abstract representations of the data, have made it possible for artificial neural networks to learn concepts such as object categories directly from raw sensory data. We use one particularly successful architecture, the deep convolutional neural network¹⁰, which uses hierarchical layers of tiled convolutional filters to learn the effects of receptive fields¹¹ inspired by Hubel and Wiesel¹² on the visual cortex, and feedforward processing in early visual cortex¹³ – thereby capturing the local spatial correlations present in images, and building up a hierarchy to natural transformations such as changes of viewpoint or scale.

We consider tasks in which the agent interacts with an environment through a sequence of observations, actions and rewards. The goal is to

agent to select actions in a fashion that maximises cumulative future reward. More formally, we use a deep convolutional neural network to approximate the optimal action-value function

$$Q^*(\mathbf{s}) = \max_a \{ \mathbb{E} [r_t + \gamma V_{t+1} + \dots + \gamma^k V_{t+k} | \mathbf{s}_t = \mathbf{s}] \}$$

which is the maximum sum of rewards r_t discounted by γ at each time-step t , achievable by a behaviour policy $\pi = \pi^*(\mathbf{s})$, after making an observation ($\mathbf{s}$) and taking an action (a) (see Methods¹⁴).

Reinforcement learning is known to be unstable or even to diverge when a nonlinear function approximator such as a neural network is used to represent the action value (also known as Q function). This instability has several causes: the correlations present in the sequence of observations, the fact that small updates to Q may significantly change the policy and therefore change the data distribution, and the correlations between the action values (Q) and the target values $r + \gamma \max_{a'} Q(\mathbf{s}', a')$.

We address these instabilities with a novel variant of Q learning, which uses two neural fits. First, we used a biologically inspired mechanism known as experience replay¹⁵, that randomises over the data, thereby removing correlations in the observation sequence and smoothing over changes in the data distribution (see below for details). Second, we used an iterative update that adapts the action-values (Q) towards target values that are not immediately updated, thereby reducing correlations with the target.

While other early methods such as the training neural networks in the reinforcement learning setting, such as so-called $TD(0)$ iteration¹⁶, these methods involve the repeated training of networks to solve on hundreds of iterations. Consequently, these methods, while our algorithm, are too inefficient to be used successfully with large neural networks. We parameterise an approximate value function $Q(\mathbf{s}, a)$ using the deep convolutional neural network shown in Fig. 1a, in which we fix the parameters (that is, weights) of the Q network at iteration i . To perform experience replay, we store the agent's experiences $\mathbf{s}_i = (\mathbf{s}_i, a_i, r_i, \mathbf{s}_{i+1})$ at each time-step i in a data set $\mathcal{D} = \{ \mathbf{s}_i, a_i, r_i, \mathbf{s}_{i+1} \}$. During learning, we apply Q learning updates, on samples from this minibatch, to the parameters (W) of the Q network ($Q(\mathbf{s}, a) = W \cdot \mathbf{f}(\mathbf{s})$), where $\mathbf{f}(\mathbf{s})$ is the output of the neural network (Q), which is used to compute the target at iteration i . The target network parameters W' are only updated with the Q network parameters (W) every C steps and are held fixed between individual updates (see Methods).

To reduce overfitting, we took advantage of the Atari 2600 platform, which offers a DQN agent the best of (C, W) designed to be

Figure 1 | Computer games. Mnih et al. have designed an artificial intelligence system, which is a deep Q network (DQN), that can learn to play 49 classic games using machine learning.

The DQN agent receives a sequence of raw game-screen images (unprocessed and unannotated), and through multiple actions it can make the consequences on the game-screen score that are taken and followed by the best possible course of subsequent actions. The first five frames of the DQN

NEWS & VIEWS

ARTIFICIAL INTELLIGENCE

Learning to see and act

An artificial-intelligence system uses machine learning from massive training sets to teach itself to play 49 classic computer games, demonstrating that it can adapt to a variety of tasks. See Letter p.29

BERNARD BOLDUCQ

Improvements in our ability to process large amounts of data have led to progress in many areas of science, not least artificial intelligence (AI). With advances in machine learning has come the development of machines that can learn intelligent behaviour directly from data, rather than being explicitly programmed to exhibit such behaviour. For instance, the advent of 'big data' has resulted in systems that can recognise objects or sounds with considerable precision. On page 329 of this issue, Mnih et al. describe an agent that uses large data sets to teach itself how to play 49 classic Atari 2600 computer games by looking at the pixels and learning actions that increase the game score. It beat a professional human player in many instances – a remarkable achievement of the progress being made in AI. In machine learning, systems are trained to infer patterns from observational data. A particularly simple type of pattern, a mapping between input and output, can be learnt through a process called supervised learning. A supervised-learning system is given training data consisting of example inputs and the corresponding outputs, and comes up with a

model to explain those data (a process called function approximation). It does this by choosing from a class of models specified by the system's designer. Designing this class is an art: its size and complexity should reflect the amount of training data available, and its content should reflect 'prior knowledge' that the designer of the system considers useful for the problem at hand. If all this is done well, the inferred model will then apply not only to the training set, but also to other data that adhere to the same underlying pattern. The rapid growth of data sets means that machine learning can now use complete model classes and tackle highly non-trivial inference problems. Such problems are usually characterised by several factors: the data are multidimensional, the underlying pattern is complex (the instance, it might be nonlinear or changeable) and the designer has only weak prior knowledge about the problem – in particular, a mechanistic understanding is lacking. The human brain repeatedly solves non-trivial inference problems as we go about our daily lives, integrating high-dimensional sensory data to determine how best to control all the muscles of the body that supervised learning is clearly unable to handle, yet because

we often learn without a 'supervisor' (defining the outputs of a hypothetical input-output function), 'reinforcement' has a central role in learning behaviours from weaker supervision. Machine learning adopted this idea to develop reinforcement learning algorithms, in which supervision takes the form of a numerical reward signal, and the goal is for the system to learn a policy that, given the current state, determines which action to take to maximise an accumulated future reward.

Mnih et al. use a form of reinforcement learning known as Q learning¹ to teach systems to play a set of 49 vintage video games, learning how to increase the game score as a numerical reward. Q -learning (Q for 'quality') represents the accumulated future reward, Q , if an agent in the system first performs action a , and subsequently follows an optimal policy. The system tries to approximate Q by using an artificial neural network – a function approximator loosely inspired by biological neural networks – called a deep Q network (DQN). The DQN input (the pixels from four consecutive game screens) is processed by connected hidden layers of computations, which extract more and more specialised visual features to help approximate the complex



Figure 1 | Computer games. Mnih et al. have designed an artificial intelligence system, which is a deep Q network (DQN), that can learn to play 49 classic games using machine learning. The DQN agent receives a sequence of raw game-screen images (unprocessed and unannotated), and through multiple actions it can make the consequences on the game-screen score that are taken and followed by the best possible course of subsequent actions. The first five frames of the DQN

analyse the pixels of the game screen and extract information from those data and from specialised visual features (image concepts) hidden within them. The connected hidden layers predict the value of a chosen future state. The feature is then subject – the action taken by the DQN. The resulting action depends on the specific game the system is playing; everything else is the same in each of the 49 games.

¹Google DeepMind, 1 New Street Square, London EC4A 3DF, UK.
²Google Research, Mountain View, CA, USA

Deep Reinforcement Learning

- Deep reinforcement learning
- Learn Q-values with deep neural networks

A tutorial on deep reinforcement learning

<https://wiki.pathmind.com/deep-reinforcement-learning>